MTH8302 - Devoir 1: Rappel Mathématique

Leonard Excoffier (2085276)

Hiver 2026

Table des matières

Problème 1 : Gradients et Hessiennes **4**
Question 1 4
Question 2 4
Question 3 4
Question 4 4
Question 5 5
Question 6 5

Problème 2 : Optimisation, Descentes de Gradient vs Newton et analyse spectrale **6**
a) Gradient 6
b) Hessienne 6
c) Comportement près de la frontière 6
d) Modèle quadratique local 7
e) Implémentation Python 7
f) Visualisation et interprétation 11
g) Analyse spectrale 12

Problème 3 : Estimation par la Méthode du Maximum de Vraisemblance et Probabilités des Estimateurs **14**
a) Estimateur du maximum de vraisemblance de λ 14
b) Biais de l'estimateur 14
c) Consistance de l'estimateur 14
d) Application numérique 15

Problème 4 : Test F de Fisher et Intervalle de Confiance **17**
a) Test F de Fisher 17
b) Visualisation 17
c) Intervalle de confiance 17
d) Code, sortie et interprétation 18

Problème 5 : Loi des Grands Nombres et Théorème Central Limite **20**
Question 1 20
Question 2 21
Question 3 23

Introduction

Le dépôt GitHub [1] contient le code complet, structuré et prêt à être exécuté. Tous les scripts Python se trouvent dans `src/` et les sorties (figures, fichiers texte) dans `output/`. Le script `run.sh` exécute l'ensemble des scripts puis compile le rapport automatiquement. Le PDF compilé est généré à la racine du projet.

Problème 1 : Gradients et Hessiennes

Question 1

Soit $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$ avec $\mathbf{A}$ symétrique.
Réponse : (b) $\nabla_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$.
Justification. On applique les règles de dérivation matricielle terme à terme :
— Forme quadratique : $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x} = 2\mathbf{A}\mathbf{x}$ car $\mathbf{A}^\top = \mathbf{A}$.
— Terme linéaire : $\frac{\partial}{\partial \mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$ (dérivée de $\mathbf{b}^\top \mathbf{A}\mathbf{x}$ avec $\mathbf{A} = \mathbf{I}$).
— Constante : $\frac{\partial c}{\partial \mathbf{x}} = \mathbf{0}$.
D'où

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \tfrac{1}{2} \cdot 2\mathbf{A}\mathbf{x} + \mathbf{b} = \boxed{\mathbf{A}\mathbf{x} + \mathbf{b}}.$$

Question 2

Même fonction $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$.
Réponse : (a) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}$.
Justification. Le gradient est $\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$. On dérive une seconde fois :

$$\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}}(\mathbf{A}\mathbf{x} + \mathbf{b}) = \boxed{\mathbf{A}}.$$

Question 3

Soit $f(\mathbf{x}) = e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}$ avec $\mathbf{A}$ symétrique.
Réponse : (b) $\nabla f(\mathbf{x}) = 2e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\mathbf{A}\mathbf{x}$.
Justification. On pose $h(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$ de sorte que $f = e^h$. Par la règle de la dérivée en chaîne :

$$\nabla f = e^h \, \nabla h.$$

Or $\nabla h = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x} = 2\mathbf{A}\mathbf{x}$ ($\mathbf{A}$ symétrique), d'où

$$\nabla f(\mathbf{x}) = \boxed{2e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\,\mathbf{A}\mathbf{x}}.$$

Question 4

Même fonction $f(\mathbf{x}) = e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}$.
Réponse : (a) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}(2\mathbf{A} + 4\mathbf{A}\mathbf{x}\mathbf{x}^\top \mathbf{A})$.
Justification. On a $\nabla f(\mathbf{x}) = 2e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\,\mathbf{A}\mathbf{x}$. La Hessienne est le Jacobien du gradient. Par la règle du produit matriciel appliquée à $\mathbf{g}(\mathbf{x}) = u(\mathbf{x})\,\mathbf{v}(\mathbf{x})$ avec $u = 2e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}$ (scalaire) et $\mathbf{v} = \mathbf{A}\mathbf{x}$ (vecteur) :

$$\frac{\partial \mathbf{g}}{\partial \mathbf{x}} = \mathbf{v}\,\frac{\partial u}{\partial \mathbf{x}}^\top + u\,\frac{\partial \mathbf{v}}{\partial \mathbf{x}}.$$

On a $\frac{\partial u}{\partial \mathbf{x}} = 2u\,\mathbf{A}\mathbf{x}$ (d'après Q3 sans le facteur 2 initial, soit $4e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\mathbf{A}\mathbf{x}$) et $\frac{\partial \mathbf{v}}{\partial \mathbf{x}} = \mathbf{A}$. Donc :

$$\nabla^2 f = (\mathbf{A}\mathbf{x})(4e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\mathbf{A}\mathbf{x})^\top + 2e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\mathbf{A} = e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}\big(4\mathbf{A}\mathbf{x}\mathbf{x}^\top \mathbf{A} + 2\mathbf{A}\big).$$

$$\boxed{\nabla^2_{\mathbf{x}} f(\mathbf{x}) = e^{\mathbf{x}^\top \mathbf{A}\mathbf{x}}(2\mathbf{A} + 4\mathbf{A}\mathbf{x}\mathbf{x}^\top \mathbf{A})}.$$

Question 5

Soit $f(\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 = (\mathbf{A}\mathbf{x})^\top(\mathbf{A}\mathbf{x}) = \mathbf{x}^\top\mathbf{A}^\top\mathbf{A}\mathbf{x}$.

Réponse : (a) $\nabla_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}^\top\mathbf{A}\mathbf{x}$.

Justification. Posons $\mathbf{B} = \mathbf{A}^\top\mathbf{A}$. La matrice $\mathbf{B}$ est symétrique car $\mathbf{B}^\top = (\mathbf{A}^\top\mathbf{A})^\top = \mathbf{A}^\top\mathbf{A} = \mathbf{B}$. Alors $f(\mathbf{x}) = \mathbf{x}^\top\mathbf{B}\mathbf{x}$ et

$$\nabla f = (\mathbf{B} + \mathbf{B}^\top)\mathbf{x} = 2\mathbf{B}\mathbf{x} = \boxed{2\mathbf{A}^\top\mathbf{A}\mathbf{x}}.$$

Question 6

Même fonction $f(\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 = \mathbf{x}^\top\mathbf{A}^\top\mathbf{A}\mathbf{x}$.

Réponse : (a) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}^\top\mathbf{A}$.

Justification. Le gradient est $\nabla f(\mathbf{x}) = 2\mathbf{A}^\top\mathbf{A}\mathbf{x}$. On dérive une seconde fois :

$$\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}}(2\mathbf{A}^\top\mathbf{A}\mathbf{x}) = \boxed{2\mathbf{A}^\top\mathbf{A}}.$$

Problème 2 : Optimisation, Descentes de Gradient vs Newton et analyse spectrale

On considère la fonction définie sur $x_1 > 0$, $x_2 > 0$:

$$f(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x} - \mu\big(\log(x_1) + \log(x_2)\big),$$

avec $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\mu = 0{,}1$, $\mathbf{x}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

a) Gradient

Réponse : (ii) $\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} - \mu \begin{pmatrix} 1/x_1 \\ 1/x_2 \end{pmatrix}$.

Justification. On dérive chaque terme :

— $\frac{\partial}{\partial \mathbf{x}}\big(\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x}\big) = \mathbf{A}\mathbf{x}$ ($\mathbf{A}$ symétrique).

— $\frac{\partial}{\partial \mathbf{x}}\big(-\mathbf{b}^\top \mathbf{x}\big) = -\mathbf{b}$.

— $\frac{\partial}{\partial x_i}\big(-\mu \log x_i\big) = -\mu/x_i$.

D'où $\nabla f(\mathbf{x}) = \boxed{\mathbf{A}\mathbf{x} - \mathbf{b} - \mu \begin{pmatrix} 1/x_1 \\ 1/x_2 \end{pmatrix}}$.

b) Hessienne

Réponse : (ii) $H(\mathbf{x}) = \mathbf{A} + \mu \operatorname{diag}\left(\frac{1}{x_1^2}, \frac{1}{x_2^2}\right)$.

Justification. On dérive le gradient composante par composante :

— $\frac{\partial}{\partial \mathbf{x}}(\mathbf{A}\mathbf{x} - \mathbf{b}) = \mathbf{A}$.

— $\frac{\partial}{\partial x_i}\big(-\mu/x_i\big) = \mu/x_i^2$ sur la diagonale, 0 ailleurs.

D'où $H(\mathbf{x}) = \boxed{\mathbf{A} + \mu \operatorname{diag}\left(\frac{1}{x_1^2}, \frac{1}{x_2^2}\right)}$.

c) Comportement près de la frontière

Réponse : (iv) Les trois affirmations sont vraies.

Justification. Quand $x_1 \to 0^+$:

(i) $-\mu \log(x_1) \to +\infty$, donc $f(\mathbf{x}) \to +\infty$. **Vrai** : la barrière logarithmique empêche $\mathbf{x}$ de sortir du domaine.

(ii) Le gradient contient $-\mu/x_1 \to -\infty$ en magnitude. **Vrai** : le gradient repousse $\mathbf{x}$ loin de la frontière.

(iii) La Hessienne contient $\mu/x_1^2 \to +\infty$, augmentant la courbure locale. **Vrai** : la fonction devient très « raide » près de la frontière.

d) Modèle quadratique local

Réponse : (ii) $H\mathbf{p} = -\mathbf{g}$.
Justification. Le modèle quadratique est :

$$f(\mathbf{x}_k + \mathbf{p}) \approx f(\mathbf{x}_k) + \mathbf{g}^\top \mathbf{p} + \tfrac{1}{2}\mathbf{p}^\top H\mathbf{p}.$$

On minimise par rapport à $\mathbf{p}$ en annulant le gradient :

$$\frac{\partial}{\partial \mathbf{p}}\left(\mathbf{g}^\top \mathbf{p} + \tfrac{1}{2}\mathbf{p}^\top H\mathbf{p}\right) = \mathbf{g} + H\mathbf{p} = \mathbf{0} \quad \Longrightarrow \quad \boxed{H\mathbf{p} = -\mathbf{g}}.$$

C'est l'équation de Newton.

e) Implémentation Python

Le code source complet est fourni au listing 1 et la sortie au listing 2.

```python
import numpy as np
import matplotlib.pyplot as plt

# Parametres
A = np.array([[4.0, 1.0], [1.0, 3.0]])
b = np.array([1.0, 2.0])
mu = 0.1


def f(x):
    x1, x2 = x[0], x[1]
    quad = 0.5 * x @ A @ x - b @ x
    barrier = -mu * (np.log(x1) + np.log(x2))
    return quad + barrier


def grad_f(x):
    g = A @ x - b - mu * np.array([1.0 / x[0], 1.0 / x[1]])
    return g


def H_f(x):
    H = A + mu * np.diag([1.0 / x[0] ** 2, 1.0 / x[1] ** 2])
    return H


def backtracking_to_stay_positive(x, step, alpha0=1.0):
    alpha = alpha0
    while np.any(x + alpha * step <= 0):
        alpha *= 0.5
    return alpha
```

```python


def run_gd(x0, eta=0.05, tol=1e-6, max_iter=2000):
    x = x0.astype(float).copy()
    hist = [x.copy()]
    for _ in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol:
            break
        step = -eta * g
        alpha = backtracking_to_stay_positive(x, step)
        x = x + alpha * step
        hist.append(x.copy())
    return np.array(hist)


def run_newton(x0, tol=1e-6, max_iter=100):
    x = x0.astype(float).copy()
    hist = [x.copy()]
    for _ in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol:
            break
        H = H_f(x)
        p = np.linalg.solve(H, -g)
        alpha = backtracking_to_stay_positive(x, p)
        x = x + alpha * p
        hist.append(x.copy())
    return np.array(hist)


def spectral_analysis(H):
    eigvals, eigvecs = np.linalg.eigh(H)
    lam_min, lam_max = eigvals[0], eigvals[-1]
    v_min, v_max = eigvecs[:, 0], eigvecs[:, -1]
    return lam_min, v_min, lam_max, v_max, eigvals, eigvecs


def main():
    x0 = np.array([1.0, 1.0])
    hist_gd = run_gd(x0, eta=0.05)
    hist_nt = run_newton(x0)

    # Grille pour contours
    x1 = np.linspace(0.1, 3.0, 200)
    x2 = np.linspace(0.1, 3.0, 200)
    X1, X2 = np.meshgrid(x1, x2)
```

```python
    Z = np.zeros_like(X1)
    for i in range(X1.shape[0]):
        for j in range(X1.shape[1]):
            Z[i, j] = f(np.array([X1[i, j], X2[i, j]]))

    # Figure 1: Courbes de niveau et trajectoires
    plt.figure(figsize=(7, 6))
    plt.contour(X1, X2, Z, levels=25)
    plt.plot(hist_gd[:, 0], hist_gd[:, 1], marker="o", markersize=2, label="GD")
    plt.plot(hist_nt[:, 0], hist_nt[:, 1], marker="o", markersize=2, label="Newton")
    plt.scatter([x0[0]], [x0[1]], s=60, label="x0")
    plt.xlabel(r"$x_1$")
    plt.ylabel(r"$x_2$")
    plt.legend()
    plt.title("Courbes de niveau et trajectoires")
    plt.tight_layout()
    plt.savefig("output/2_contour.png", dpi=200)
    plt.close()

    # Figure 2: Norme du gradient
    plt.figure(figsize=(7, 4))
    plt.plot([np.linalg.norm(grad_f(x)) for x in hist_gd], label="GD")
    plt.plot([np.linalg.norm(grad_f(x)) for x in hist_nt], label="Newton")
    plt.yscale("log")
    plt.xlabel(r"it\'eration $k$")
    plt.ylabel(r"$\|\nabla f(x_k)\|$")
    plt.legend()
    plt.title("Evolution de la norme du gradient")
    plt.tight_layout()
    plt.savefig("output/2_gradient.png", dpi=200)
    plt.close()

    # Sortie texte
    lines = []
    lines.append(f"Descente de gradient: {len(hist_gd)-1} iterations")
    lines.append(f"  x* = [{hist_gd[-1, 0]:.6f}, {hist_gd[-1, 1]:.6f}]")
    lines.append(f"  f(x*) = {f(hist_gd[-1]):.6f}")
    lines.append(f"  ||grad|| = {np.linalg.norm(grad_f(hist_gd[-1])):.2e}")
    lines.append(f"Newton: {len(hist_nt)-1} iterations")
    lines.append(f"  x* = [{hist_nt[-1, 0]:.6f}, {hist_nt[-1, 1]:.6f}]")
    lines.append(f"  f(x*) = {f(hist_nt[-1]):.6f}")
    lines.append(f"  ||grad|| = {np.linalg.norm(grad_f(hist_nt[-1])):.2e}")

    # Analyse spectrale
    pts = [("x^(1) = (1,1)", np.array([1.0, 1.0])), ("x^(2) = (0.2,0.2)",
```

```python
    np.array([0.2, 0.2]))]
    for name, xp in pts:
        H = H_f(xp)
        lam_min, v_min, lam_max, v_max, _, _ = spectral_analysis(H)
        kappa = lam_max / lam_min
        lines.append("=" * 60)
        lines.append(f"Point {name}")
        lines.append(f"H(x) =\n{H}")
        lines.append(f"lambda_min = {lam_min:.6f}, v_min = {v_min}")
        lines.append(f"lambda_max = {lam_max:.6f}, v_max = {v_max}")
        lines.append(f"kappa = {kappa:.6f}")

    with open("output/2.txt", "w", encoding="utf-8") as fout:
        fout.write("\n".join(lines) + "\n")


if __name__ == "__main__":
    main()
```

Listing 1 – Code Python pour l'optimisation (src/2.py)

```
Descente de gradient: 67 iterations
  x* = [0.208009, 0.648714]
  f(x*) = -0.452425
  ||grad|| = 9.59e-07
Newton: 5 iterations
  x* = [0.208009, 0.648714]
  f(x*) = -0.452425
  ||grad|| = 8.06e-12
============================================================
Point x^(1) = (1,1)
H(x) =
[[4.1 1. ]
 [1.  3.1]]
lambda_min = 2.481966, v_min = [ 0.52573111 -0.85065081]
lambda_max = 4.718034, v_max = [-0.85065081 -0.52573111]
kappa = 1.900926
============================================================
Point x^(2) = (0.2,0.2)
H(x) =
[[6.5 1. ]
 [1.  5.5]]
lambda_min = 4.881966, v_min = [ 0.52573111 -0.85065081]
lambda_max = 7.118034, v_max = [-0.85065081 -0.52573111]
kappa = 1.458026
```

Listing 2 – Sortie du programme (output/2.txt)

f) Visualisation et interprétation

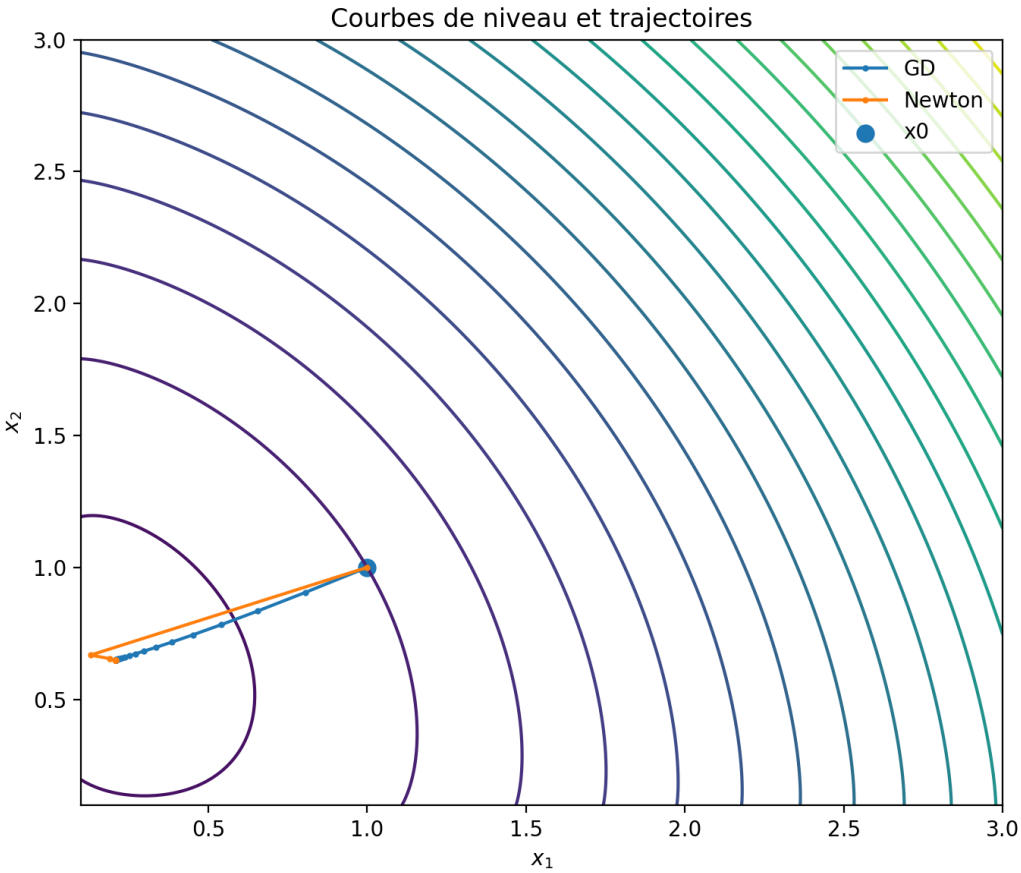

FIGURE 1 – Courbes de niveau de f et trajectoires des itérés pour GD et Newton (output/2_contour.png).

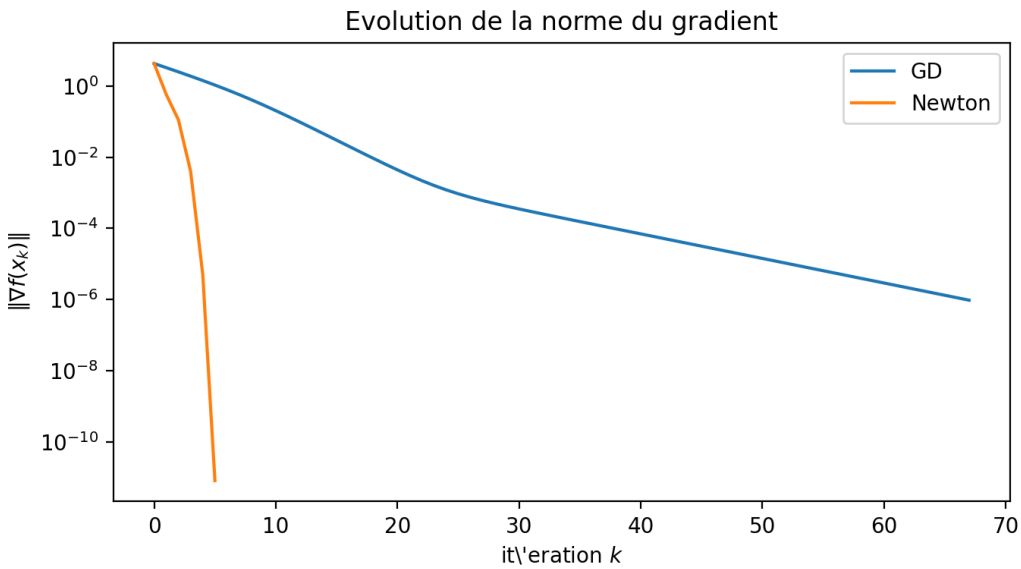

FIGURE 2 – Évolution de $\|\nabla f(\mathbf{x}_k)\|$ en fonction de k (output/2_gradient.png).

Interprétation. La méthode de Newton converge en **5 itérations** contre **67 pour GD**. Newton converge beaucoup plus vite car elle exploite l'information de courbure (Hessienne) pour calculer la direction de descente optimale. La descente de gradient, avec un pas constant $\eta = 0{,}05$, suit une trajectoire en zigzag typique, car elle ne tient pas compte de l'anisotropie de la Hessienne. Newton ajuste implicitement le pas et la direction via $H^{-1}\mathbf{g}$, ce qui lui permet d'atteindre le minimum en très peu d'itérations (convergence quadratique vs linéaire).

g) Analyse spectrale

(i) Valeurs propres et vecteurs propres.
Pour $\mathbf{x}^{(1)} = (1, 1)^\top$:

$$H(\mathbf{x}^{(1)}) = \begin{pmatrix} 4{,}1 & 1 \\ 1 & 3{,}1 \end{pmatrix}, \quad \lambda_{\min} = 2{,}482, \quad \lambda_{\max} = 4{,}718.$$

Pour $\mathbf{x}^{(2)} = (0{,}2,\ 0{,}2)^\top$:

$$H(\mathbf{x}^{(2)}) = \begin{pmatrix} 6{,}5 & 1 \\ 1 & 5{,}5 \end{pmatrix}, \quad \lambda_{\min} = 4{,}882, \quad \lambda_{\max} = 7{,}118.$$

Les vecteurs propres associés sont identiques dans les deux cas : $\mathbf{v}_{\min} \approx (0{,}526,\ -0{,}851)^\top$, $\mathbf{v}_{\max} \approx (-0{,}851,\ -0{,}526)^\top$.

(ii) Signification de $\kappa(\mathbf{x})$. Le nombre de conditionnement $\kappa = \lambda_{\max}/\lambda_{\min}$ mesure l'**anisotropie** de la Hessienne. Un κ élevé signifie que les courbes de niveau sont très allongées (ellipses aplaties), ce qui ralentit la descente de gradient qui zigzague entre les parois de la « vallée ». Un κ proche de 1 signifie des courbes de niveau quasi circulaires, favorable à GD.

(iii) Conditionnement aux deux points.

$$\kappa(\mathbf{x}^{(1)}) = \frac{4{,}718}{2{,}482} \approx 1{,}901, \qquad \kappa(\mathbf{x}^{(2)}) = \frac{7{,}118}{4{,}882} \approx 1{,}458.$$

Les deux valeurs sont proches de 1, ce qui indique un problème **bien conditionné** aux deux points. Le conditionnement est même meilleur en $\mathbf{x}^{(2)}$ ($\kappa \approx 1{,}46$) qu'en $\mathbf{x}^{(1)}$ ($\kappa \approx 1{,}90$), malgré la proximité de la frontière. Cela explique la convergence raisonnablement rapide de GD (67 itérations) pour ce problème. Près de la frontière, la barrière logarithmique ajoute de la courbure (termes μ/x_i^2), ce qui augmente les deux valeurs propres et peut améliorer le conditionnement.

Problème 3 : Estimation par la Méthode du Maximum de Vraisemblance et Probabilités des Estimateurs

Soit $X_1, X_2, \ldots, X_n$ un échantillon aléatoire issu d'une **loi de Poisson** de paramètre λ :

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \ldots$$

a) Estimateur du maximum de vraisemblance de λ

Fonction de vraisemblance.

$$L(\lambda; \mathbf{X}) = \prod_{i=1}^{n} \frac{\lambda^{X_i} e^{-\lambda}}{X_i!} = \frac{\lambda^{\sum_{i=1}^{n} X_i} e^{-n\lambda}}{\prod_{i=1}^{n} X_i!}.$$

Log-vraisemblance.

$$\ell(\lambda) = \log L(\lambda) = \left(\sum_{i=1}^{n} X_i \right) \log \lambda - n\lambda - \sum_{i=1}^{n} \log(X_i!).$$

Maximisation. On dérive par rapport à λ et on annule :

$$\frac{d\ell}{d\lambda} = \frac{\sum_{i=1}^{n} X_i}{\lambda} - n = 0 \quad \Longrightarrow \quad \boxed{\hat{\lambda}_{\text{EMV}} = \bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i.}$$

Vérification (condition du second ordre). $\dfrac{d^2\ell}{d\lambda^2} = -\dfrac{\sum_{i=1}^{n} X_i}{\lambda^2} < 0$, donc $\hat{\lambda}$ est bien un maximum.

b) Biais de l'estimateur

On calcule l'espérance de $\hat{\lambda}$:

$$\mathbb{E}[\hat{\lambda}] = \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^{n} X_i \right] = \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}[X_i] = \frac{1}{n} \cdot n\lambda = \lambda.$$

Le biais est $b(\hat{\lambda}) = \mathbb{E}[\hat{\lambda}] - \lambda = 0$. L'estimateur $\hat{\lambda} = \bar{X}$ est donc **non biaisé**.

c) Consistance de l'estimateur

Pour une variable de Poisson, $\text{Var}(X_i) = \lambda$. On a donc :

$$\text{Var}(\hat{\lambda}) = \text{Var}(\bar{X}) = \frac{1}{n^2} \sum_{i=1}^{n} \text{Var}(X_i) = \frac{n\lambda}{n^2} = \frac{\lambda}{n}.$$

L'erreur quadratique moyenne est :

$$EQM(\hat{\lambda}_n) = \text{Var}(\hat{\lambda}_n) + [b(\hat{\lambda}_n)]^2 = \frac{\lambda}{n} + 0 = \frac{\lambda}{n}.$$

Comme $\lim_{n \to \infty} EQM(\hat{\lambda}_n) = \lim_{n \to \infty} \dfrac{\lambda}{n} = 0$, l'estimateur $\hat{\lambda}$ est **consistant**.

d) Application numérique

On dispose des fréquences observées suivantes :

x	0	1	2	3	4	5
$f_X(x)$	7	14	12	13	6	3

(i) EMV de λ. Le nombre total d'observations est $n = 7 + 14 + 12 + 13 + 6 + 3 = 55$ et

$$\sum_{i=1}^{n} X_i = 0(7) + 1(14) + 2(12) + 3(13) + 4(6) + 5(3) = 116.$$

D'où

$$\hat{\lambda} = \bar{X} = \frac{116}{55} \approx 2{,}1091.$$

(ii) Estimation de $P(X = 2)$. On substitue $\hat{\lambda}$ dans la loi de Poisson :

$$\hat{P}(X = 2) = \frac{\hat{\lambda}^2 \, e^{-\hat{\lambda}}}{2!} = \frac{(116/55)^2 \, e^{-116/55}}{2} \approx 0{,}2699.$$

Code et sortie. Le code source est fourni au listing 3 et la sortie au listing 4.

```python
import numpy as np
from scipy.stats import poisson


def main():
    # Frequences observees
    x_vals = np.array([0, 1, 2, 3, 4, 5])
    freqs = np.array([7, 14, 12, 13, 6, 3])

    n = freqs.sum()
    sum_xi = (x_vals * freqs).sum()

    # EMV de lambda
    lambda_hat = sum_xi / n

    # Estimation de P(X = 2)
    p_x2 = poisson.pmf(2, lambda_hat)

    output_lines = [
        f"n = {n}",
        f"sum(x_i) = {sum_xi}",
        f"lambda_hat (EMV) = {lambda_hat:.6f}",
        f"P(X = 2) estimee = {p_x2:.6f}",
    ]
    with open("output/3.txt", "w", encoding="utf-8") as f_out:
        f_out.write("\n".join(output_lines) + "\n")
```

```python


if __name__ == "__main__":
    main()
```

Listing 3 – Code Python pour l'EMV de la loi de Poisson (src/3.py)

```
n = 55
sum(x_i) = 116
lambda_hat (EMV) = 2.109091
P(X = 2) estimee = 0.269895
```

Listing 4 – Sortie du programme (output/3.txt)

Problème 4 : Test F de Fisher et Intervalle de Confiance

a) Test F de Fisher

Hypothèses. On teste $H_0 : \sigma_A^2 = \sigma_B^2$ contre $H_1 : \sigma_A^2 \neq \sigma_B^2$ (test bilatéral), avec $\alpha = 0.05$.

Les variances d'échantillon calculées sont $s_A^2 = 16.988889$ et $s_B^2 = 10.266667$, d'où la statistique $F_{\text{stat}} = s_A^2/s_B^2 = 1.654762$. Les valeurs critiques pour $(\nu_1, \nu_2) = (9, 9)$ sont $F_{0.025} = 0.248386$ et $F_{0.975} = 4.025994$, et la p-valeur vaut 0.464710. Comme F_{stat} est dans la région d'acceptation et $p > 0.05$, on ne rejette pas H_0 : il n'y a pas d'évidence d'une différence de variances au seuil de 5%.

b) Visualisation

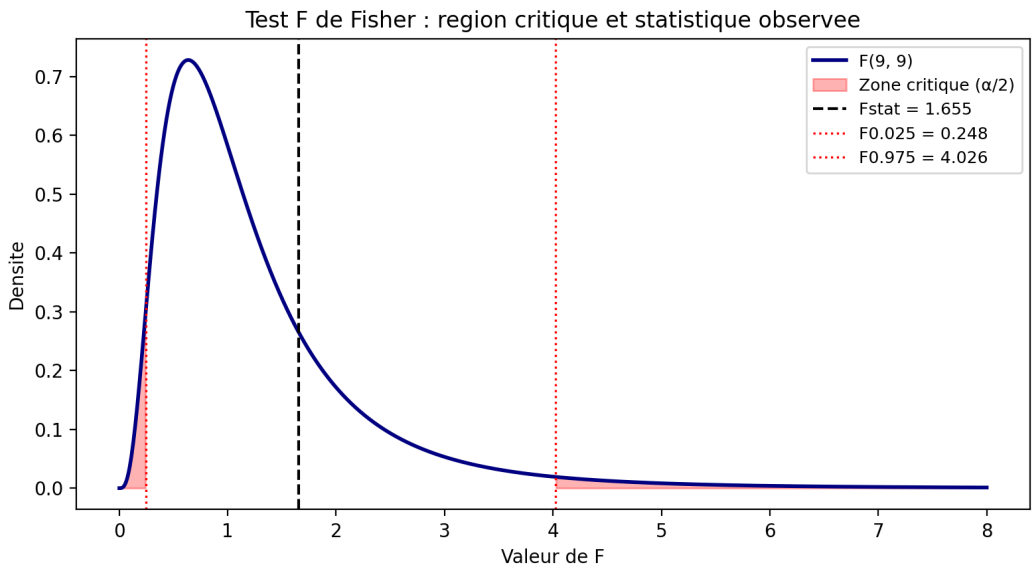

FIGURE 3 – Distribution $F(9, 9)$ avec les zones critiques et la statistique observée F_{stat} (output/4.png).

La figure 3 montre la densité $F(9, 9)$, les régions critiques (en rouge) et la position de F_{stat}, qui se situe hors des zones de rejet.

c) Intervalle de confiance

Intervalle de confiance à 95% pour σ_A^2/σ_B^2. En utilisant les quantiles $F_{0.025}$ et $F_{0.975}$, l'intervalle est

$$\left[\frac{F_{\text{stat}}}{F_{0.975}}, \frac{F_{\text{stat}}}{F_{0.025}}\right] = [0.411019,\ 6.662062].$$

d) Code, sortie et interprétation

Code et sortie. Le code source est fourni au listing 5 et la sortie correspondante au listing 6.

```python
import numpy as np
from scipy.stats import f
import matplotlib.pyplot as plt


def main():
    # Donnees
    group_a = np.array([72, 75, 78, 69, 82, 77, 71, 74, 80, 73], dtype=float)
    group_b = np.array([78, 81, 85, 79, 88, 84, 82, 80, 86, 83], dtype=float)

    n_a = len(group_a)
    n_b = len(group_b)
    df1 = n_a - 1
    df2 = n_b - 1
    alpha = 0.05

    var_a = group_a.var(ddof=1)
    var_b = group_b.var(ddof=1)
    f_stat = var_a / var_b

    f_lower = f.ppf(alpha / 2, df1, df2)
    f_upper = f.ppf(1 - alpha / 2, df1, df2)

    p_value = 2 * min(f.cdf(f_stat, df1, df2), 1 - f.cdf(f_stat, df1, df2))

    ci_lower = f_stat / f_upper
    ci_upper = f_stat / f_lower

    output_lines = [
        f"Variance A (s^2_A) = {var_a:.6f}",
        f"Variance B (s^2_B) = {var_b:.6f}",
        f"F_stat = {f_stat:.6f}",
        f"F_0.025 = {f_lower:.6f}",
        f"F_0.975 = {f_upper:.6f}",
        f"p-value = {p_value:.6f}",
        f"IC 95% du ratio (sigma_A^2 / sigma_B^2) = [{ci_lower:.6f}, {ci_upper:.6f}]",
    ]
    with open("output/4.txt", "w", encoding="utf-8") as f_out:
        f_out.write("\n".join(output_lines) + "\n")

    # Trace de la distribution F et region critique
    x_max = max(8.0, f_upper * 1.2)
```

```python
    x = np.linspace(0.001, x_max, 1000)
    y = f.pdf(x, df1, df2)

    plt.figure(figsize=(8, 4.5))
    plt.plot(x, y, color="navy", lw=2, label=f"F({df1}, {df2})")
    plt.fill_between(x, y, where=(x <= f_lower), color="red", alpha=0.3, label="Zone critique (\u03b1/2)")  # type: ignore
    plt.fill_between(x, y, where=(x >= f_upper), color="red", alpha=0.3)  # type: ignore
    plt.axvline(f_stat, color="black", linestyle="--", lw=1.5, label=f"Fstat = {f_stat:.3f}")
    plt.axvline(f_lower, color="red", linestyle=":", lw=1.2, label=f"F0.025 = {f_lower:.3f}")  # type: ignore
    plt.axvline(f_upper, color="red", linestyle=":", lw=1.2, label=f"F0.975 = {f_upper:.3f}")  # type: ignore
    plt.title("Test F de Fisher : region critique et statistique observee")
    plt.xlabel("Valeur de F")
    plt.ylabel("Densite")
    plt.legend(fontsize=9)
    plt.tight_layout()
    plt.savefig("output/4.png", dpi=200)
    plt.close()


if __name__ == "__main__":
    main()
```

Listing 5 – Code Python pour le test F et l'intervalle de confiance (src/4.py)

```
Variance A (s^2_A) = 16.988889
Variance B (s^2_B) = 10.266667
F_stat = 1.654762
F_0.025 = 0.248386
F_0.975 = 4.025994
p-value = 0.464710
IC 95% du ratio (sigma_A^2 / sigma_B^2) = [0.411019, 6.662062]
```

Listing 6 – Sortie du programme (output/4.txt)

Interprétation. L'intervalle de confiance contient la valeur 1, donc un ratio $\sigma_A^2/\sigma_B^2 = 1$ est plausible. Cette conclusion est cohérente avec le test F qui ne rejette pas H_0.

Problème 5 : Loi des Grands Nombres et Théorème Central Limite

On simule une variable aléatoire $X \sim \text{Bernoulli}(p = 0{,}5)$ et on étudie le comportement de la moyenne empirique $\overline{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$ lorsque n augmente.

Question 1

Influence de n sur la dispersion de $\overline{X}_n$ (loi faible des grands nombres).

On répète 2000 fois le calcul de $\overline{X}_n$ pour $n \in \{50,\ 500,\ 5000\}$ et on trace l'histogramme des moyennes obtenues.

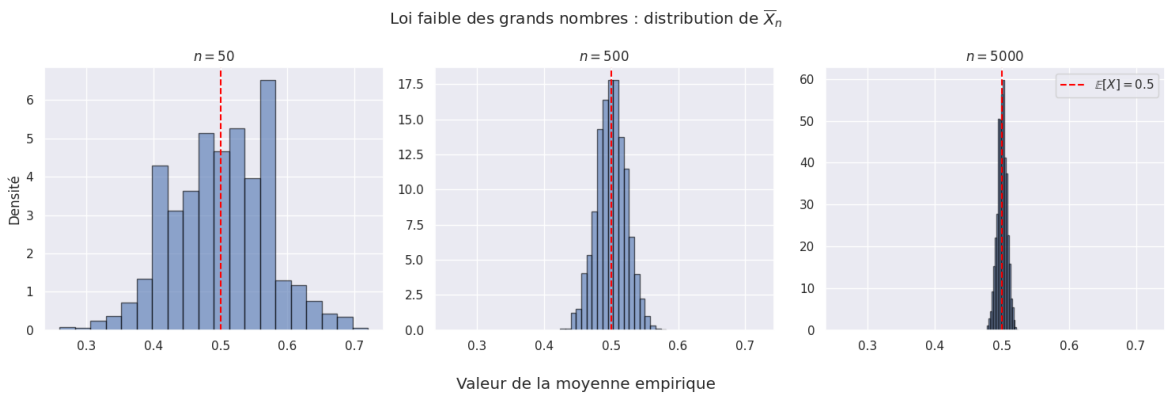

FIGURE 4 – Distribution de $\overline{X}_n$ pour $n = 50$, 500 et 5000 (2000 répétitions) (output/5_1.png).

La figure 4 montre que lorsque n augmente, la distribution de $\overline{X}_n$ se concentre de plus en plus autour de $\mathbb{E}[X] = 0{,}5$. Concrètement, la variance de $\overline{X}_n$ est $\text{Var}(\overline{X}_n) = \frac{p(1-p)}{n} = \frac{0{,}25}{n}$, qui décroît en $1/n$. Pour $n = 50$ l'histogramme est large (écart-type $\approx 0{,}071$), pour $n = 500$ il est nettement plus étroit ($\approx 0{,}022$), et pour $n = 5000$ les valeurs sont quasiment toutes égales à 0,5 ($\approx 0{,}007$). Cela illustre la **loi faible des grands nombres** : $\overline{X}_n \xrightarrow{P} \mathbb{E}[X]$ lorsque $n \to \infty$.

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


def main():
    sns.set_theme()

    np.random.seed(42)

    # Paramètres
    n_list = [50, 500, 5000]  # Nombre d'échantillons
```

```python
    num_trials = 2000  # Nombre de répétitions

    fig, axes = plt.subplots(1, len(n_list), figsize=(15, 5), sharex=True)

    for ax, n in zip(axes, n_list):
        # Génération des variables de Bernoulli
        X = np.random.binomial(1, 0.5, (num_trials, n))

        # Calcul des moyennes cumulatives
        means = np.cumsum(X, axis=1) / np.arange(1, n + 1)

        final_means = means[:, -1]  # Moyenne

        ax.hist(final_means, bins=20, density=True, alpha=0.6, color="b", edgecolor="black")
        ax.axvline(0.5, color="red", linestyle="dashed", label=r"$\mathbb{E}[X] = 0.5$")
        ax.set_title(rf"$n = {n}$")

    axes[0].set_ylabel("Densité")
    fig.supxlabel("Valeur de la moyenne empirique")
    axes[-1].legend()
    fig.suptitle(r"Loi faible des grands nombres : distribution de $\overline{X}_n$")
    fig.tight_layout()
    plt.savefig("output/5_1.png")


if __name__ == "__main__":
    main()
```

Listing 7 – Code Python - Loi faible des grands nombres (src/5_1.py)

Question 2

Trajectoires individuelles de $\overline{X}_n$ (loi forte des grands nombres).

On fixe $n = 1000$ et on trace les trajectoires de la moyenne cumulative $\overline{X}_k$ $(k = 1, \ldots, n)$ pour $N_{\text{rep}} \in \{1,\ 5,\ 10,\ 1000\}$ répétitions.

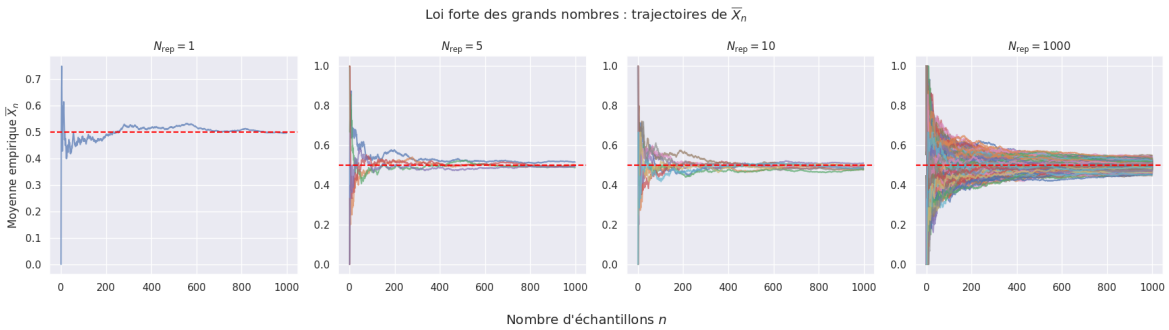

FIGURE 5 – Trajectoires de $\overline{X}_n$ en fonction de n pour différents nombres de répétitions (output/5_2.png).

La figure 5 montre que, quel que soit le nombre de répétitions, **chaque trajectoire individuelle converge vers** $\mathbb{E}[X] = 0{,}5$ lorsque n grandit. Même avec une seule trajectoire ($N_{\text{rep}} = 1$), la moyenne cumulative se stabilise autour de 0,5 pour n suffisamment grand. En augmentant le nombre de répétitions, on observe un faisceau de trajectoires qui se resserre autour de 0,5 : les fluctuations initiales sont grandes, mais elles s'amortissent toutes de la même façon. Cela illustre la **loi forte des grands nombres** : $\overline{X}_n \xrightarrow{\text{p.s.}} \mathbb{E}[X]$, c'est-à-dire que la convergence a lieu pour *presque toute* réalisation, et non seulement en probabilité.

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


def main():
    sns.set_theme()

    np.random.seed(42)

    # LOI FORTE DES GRANDS NOMBRES: Trajectoires individuelles
    # Paramètres
    n = 1000  # Nombre d'échantillons
    num_trials_list = [1, 5, 10, 1000]  # Nombre de répétitions

    fig, axes = plt.subplots(1, len(num_trials_list), figsize=(18, 5),
    sharex=True)

    for ax, num_trials in zip(axes, num_trials_list):
        # Génération des variables de Bernoulli
        X = np.random.binomial(1, 0.5, (num_trials, n))

        # Calcul des moyennes cumulatives
        means = np.cumsum(X, axis=1) / np.arange(1, n + 1)

        for i in range(num_trials):  # Tracer les trajectoires différentes
```

```python
            ax.plot(means[i, :], alpha=0.7)

        ax.axhline(0.5, color="red", linestyle="dashed", label=r"$\mathbb{E}[X] = 0.5$")
        ax.set_title(r"$N_{\mathrm{rep}} = %d$" % num_trials)
    axes[0].set_ylabel(r"Moyenne empirique $\overline{X}_n$")
    fig.supxlabel(r"Nombre d'échantillons $n$")
    fig.suptitle(r"Loi forte des grands nombres : trajectoires de $\overline{X}_n$")
    fig.tight_layout()
    plt.savefig("output/5_2.png")


if __name__ == "__main__":
    main()
```

Listing 8 – Code Python - Loi forte des grands nombres (src/5_2.py)

Question 3

Convergence vers la densité normale (théorème central limite).
On calcule $\overline{X}_n$ sur 500 répétitions pour $n \in \{100,\ 1000,\ 10\,000\}$ et on superpose la densité de la loi $\mathcal{N}\left(0{,}5,\ \frac{p(1-p)}{n}\right)$.

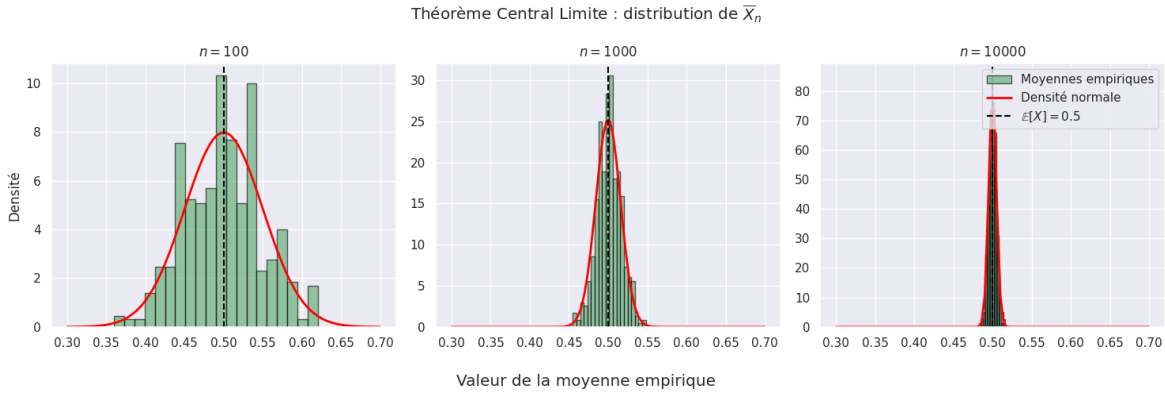

FIGURE 6 – Distribution de $\overline{X}_n$ comparée à la densité normale théorique pour $n = 100$, 1000 et 10 000 (output/5_3.png).

La figure 6 montre que l'histogramme des moyennes empiriques épouse de mieux en mieux la courbe de la densité normale (en rouge) à mesure que n augmente. Dès $n = 100$ la forme est approximativement gaussienne ; pour $n = 1000$ et surtout $n = 10\,000$ la correspondance est excellente. L'histogramme converge donc vers la densité de la loi **normale** $\mathcal{N}(\mu,\ \sigma^2/n)$ avec $\mu = p = 0{,}5$ et $\sigma^2 = p(1-p) = 0{,}25$. Cela illustre le **théorème central limite** :

$$\sqrt{n}\,\frac{\overline{X}_n - \mu}{\sigma} \xrightarrow{\mathcal{L}} \mathcal{N}(0,1).$$

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats


def main():
    sns.set_theme()

    np.random.seed(42)

    num_trials = 500
    n_list = [100, 1000, 10000]

    fig, axes = plt.subplots(1, len(n_list), figsize=(15, 5), sharex=True)

    for ax, n in zip(axes, n_list):
        X_tcl = np.random.binomial(1, 0.5, (num_trials, n))
        sample_means = np.mean(X_tcl, axis=1)

        ax.hist(sample_means, bins=20, density=True, alpha=0.6, color="g", edgecolor="black", label="Moyennes empiriques")

        # Superposition d'une distribution normale
        mu, sigma = 0.5, np.sqrt(0.5 * 0.5 / n)
        x_vals = np.linspace(0.3, 0.7, 100)
        ax.plot(x_vals, stats.norm.pdf(x_vals, mu, sigma), color="red", lw=2, label="Densité normale")

        ax.axvline(0.5, color="black", linestyle="dashed", label=r"$\mathbb{E}[X] = 0.5$")
        ax.set_title(f"$n={n}$")

    axes[0].set_ylabel("Densité")
    fig.supxlabel("Valeur de la moyenne empirique")
    axes[-1].legend()
    fig.suptitle(r"Théorème Central Limite : distribution de $\overline{X}_n$")
    fig.tight_layout()
    plt.savefig("output/5_3.png")


if __name__ == "__main__":
    main()
```

Listing 9 – Code Python - Théorème central limite (src/5_3.py)

Références

[1] Leonard Excoffier. *MTH8302 - Devoir 1 : Rappel Mathématique*. GitHub repository. `https://github.com/excoffierleonard/mth8302-devoir1.git`. Consulté le 18 février 2026.